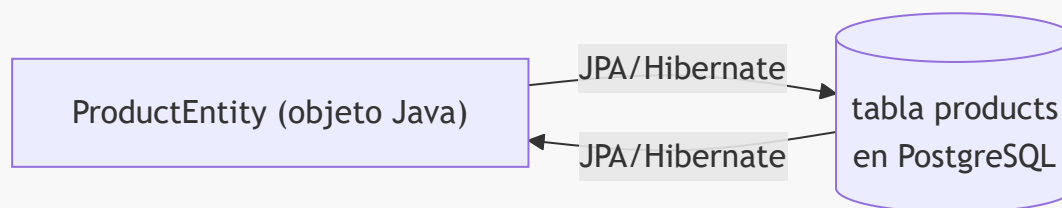


Fase 3 — Persistencia: JPA, Entities y PostgreSQL

La fase más pesada, y la que tiene una diferencia real con tu proyecto: la evaluación siembra datos con `schema.sql/data.sql`, algo que tú no has practicado.

1. Qué es un ORM y qué problema resuelve

Un **ORM** (Object-Relational Mapper) traduce entre objetos Java y filas de una tabla SQL, sin que tengas que escribir `INSERT/SELECT` a mano.



Sin ORM: abrir conexión, armar SQL, mapear `ResultSet` a objeto a mano. Con JPA: anotas la clase, Hibernate genera el SQL y hace el mapeo por ti.

2. `@Entity`, `@Table`, `@Id`, `@Column`

```
@Entity
@Table(name = "products")
public class ProductEntity extends BaseEntity {

    @Column(nullable = false, length = 150)
    private String name;

    @Column(nullable = false)
    private Double price;

    @ManyToOne(optional = false, fetch = FetchType.EAGER)
    @JoinColumn(name = "user_id", nullable = false)
    private UserEntity owner;
}
```

Anotación	Qué hace
<code>@Entity</code>	Esta clase se mapea a una tabla. Hibernate la gestiona.

Anotación	Qué hace
<code>@Table(name = "products")</code>	Nombre real de la tabla (si se omite, usa el nombre de la clase).
<code>@Id + @GeneratedValue(strategy = GenerationType.IDENTITY)</code>	Clave primaria autoincremental (delegada a PostgreSQL).
<code>@Column(nullable = false, length = 150)</code>	Restricciones a nivel de columna: <code>NOT NULL</code> , <code>VARCHAR(150)</code> .
<code>@ManyToOne / @JoinColumn</code>	Relación con otra entidad; crea la FK (<code>user_id</code>) en esta tabla.
<code>@ManyToMany + @JoinTable</code>	Relación N:N; genera una tabla intermedia (<code>product_categories</code>).

Tu proyecto usa `BaseEntity` (`@MappedSuperclass`) para no repetir `id`, `createdAt`, `updatedAt`, `isDeleted` en cada entidad:

```
@MappedSuperclass
public abstract class BaseEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    private boolean isDeleted = false;

    @PrePersist
    protected void onCreate() { this.createdAt = LocalDateTime.now(); }

    @PreUpdate
    protected void onUpdate() { this.updatedAt = LocalDateTime.now(); }
}
```

`@MappedSuperclass` **no crea tabla propia** — sus columnas se heredan dentro de la tabla de cada hija (`products`, `categories`, etc).

3. `JpaRepository` y métodos derivados

```
@Repository
public interface ProductRepository extends JpaRepository<ProductEntity, Long> {

    List<ProductEntity> findByIdAndIsDeletedFalse(Long userId);

    List<ProductEntity> findByCategoriesIdAndIsDeletedFalse(Long categoryId);
}
```

`JpaRepository<ProductEntity, Long>` ya trae gratis `save`, `findById`, `findAll`, `deleteById`. Los métodos **derivados** se generan a partir del nombre del método — Spring lo parsea y arma el SQL.

Fragmento del nombre	Se traduce a
<code>findBy</code>	<code>SELECT ... WHERE</code>
<code>OwnerId</code>	<code>owner.id = ?</code> (navega la relación <code>owner</code>)
<code>And</code>	<code>AND</code>
<code>IsDeletedFalse</code>	<code>is_deleted = false</code>
<code>existsBy...</code>	<code>SELECT COUNT(*) > 0 ...</code>

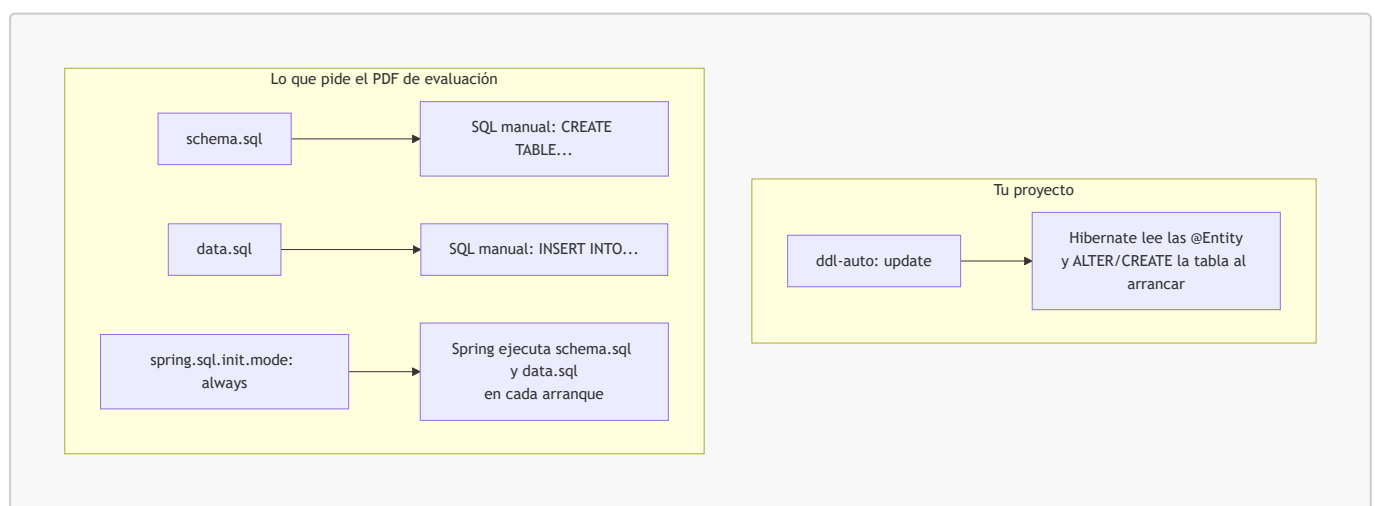
Ejemplo pedido en la evaluación: `findByActiveAndStockLessThan(boolean active, int stock)` → `WHERE active = ? AND stock < ?`. Mismo mecanismo, otro campo.

Cuando el filtro es opcional o cruza relaciones (como en `findByOwnerIdWithFilters`), tu proyecto pasa a `@Query` con JPQL:

```
@Query("""
    SELECT DISTINCT p FROM ProductEntity p
    LEFT JOIN p.categories c
    WHERE p.isDeleted = false
           AND p.owner.id = :userId
           AND (:categoryId IS NULL OR c.id = :categoryId)
""")
List<ProductEntity> findByOwnerIdWithFilters(@Param("userId") Long userId,
@Param("categoryId") Long categoryId, ...);
```

Regla: método derivado si el filtro es simple y fijo; `@Query` (JPQL) si hay condiciones opcionales (`COALESCE`, `:param IS NULL OR ...`) o joins complejos.

4. ddl-auto: update vs schema.sql + data.sql



	ddl-auto: update (tu proyecto)	schema.sql + data.sql (evaluación)
Quién crea la tabla	Hibernate, leyendo las anotaciones <code>@Entity</code>	Tú, escribiendo <code>CREATE TABLE</code> a mano
Quién siembra datos	Nadie automático (o un <code>CommandLineRunner</code>)	<code>data.sql</code> con <code>INSERT</code>
Dónde vive	<code>application-dev.yml</code> : <code>spring.jpa.hibernate.ddl-auto</code>	<code>src/main/resources/schema.sql</code> + <code>data.sql</code>
Flag necesario	—	<code>spring.sql.init.mode: always</code> (si ya hay JPA, si no se ejecuta solo)
Cuándo se usa	Desarrollo rápido, prototipo	Control total del esquema, datos reproducibles, exámenes

Tu config actual (`application-dev.yml`):

```
spring:
  jpa:
    hibernate:
      ddl-auto: ${JPA_DDL_AUTO:update}
```

Para el mecanismo que pide el PDF necesitarías, en `src/main/resources/`:

```
-- schema.sql
CREATE TABLE IF NOT EXISTS products (...);

-- data.sql
INSERT INTO products (name, price, stock) VALUES ('Mouse', 15.5, 100);
```

y en `application.yml`:

```
spring:
  sql:
    init:
      mode: always
```

Con `ddl-auto: update` + `schema.sql` activos a la vez pueden pisarse — por eso en un examen normalmente se usa `ddl-auto: none` cuando hay `schema.sql`.

5. SQL básico: `SELECT`, `WHERE`, `ORDER BY`

```
SELECT * FROM products;
SELECT * FROM products WHERE stock < 10;
SELECT * FROM products WHERE is_deleted = false ORDER BY price DESC;
```

Cada método de `ProductRepository` termina generando algo así. Ejemplo — `findActivePage` (usa `Page`, con `countQuery` para el total):

```
SELECT * FROM products WHERE is_deleted = false LIMIT 10 OFFSET 0;
SELECT COUNT(*) FROM products WHERE is_deleted = false;
```

Resumen / Chuleta

Pregunta	Respuesta corta
¿Qué problema resuelve un ORM?	Evita escribir SQL y mapear <code>ResultSet</code> a mano; traduce objetos ↔ filas.
¿Qué hace <code>@MappedSuperclass</code> ?	Comparte columnas entre entidades sin crear tabla propia.
¿Cuándo uso método derivado vs <code>@Query</code> ?	Derivado si el filtro es fijo/simple; <code>@Query</code> si hay condiciones opcionales o joins complejos.
¿Diferencia clave <code>ddl-auto</code> vs <code>schema.sql</code> ?	<code>ddl-auto</code> = Hibernate infiere el esquema de las entidades; <code>schema.sql/data.sql</code> = tú controlas el SQL exacto que se ejecuta al arrancar.

Cómo estudiarlo

Conecta con psql o pgAdmin y corre `SELECT * FROM products;` a mano mientras creas/editas productos por la API — observa el efecto en vivo. Luego arma un mini-proyecto de práctica solo para `schema.sql` + `data.sql`, ese mecanismo que aún no usas.

Práctica sugerida: Práctica 5

Checklist

- ☐ Sé qué hacen `@Entity`, `@Table`, `@Id` y `@Column`
- ☐ Sé escribir un método derivado tipo `findByActiveAndStockLessThan`
- ☐ Entiendo la diferencia entre `ddl-auto: update` y `schema.sql/data.sql`
- ☐ Corrí un `SELECT` a mano mientras probaba la API